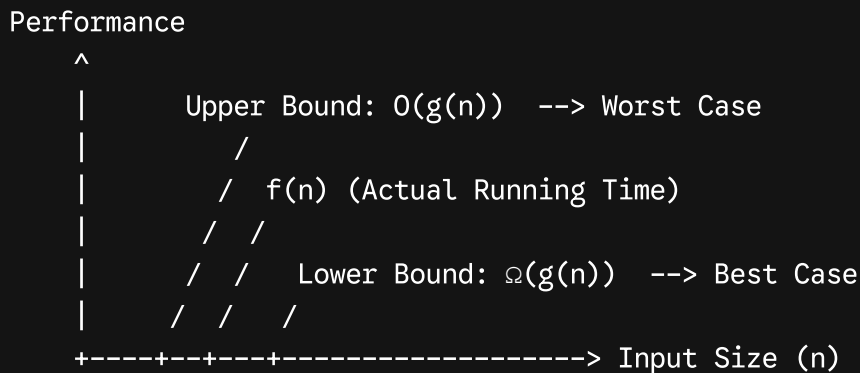


Asymptotic notation describes the execution time or space complexity of an algorithm as the input size grows toward infinity ($n \rightarrow \infty$). It discards low-order terms and constant factors to focus on the growth rate.

The Three Main Notations



1. Big O Notation ($\mathcal{O}$): Upper Bound (Worst Case)

- Definition:** $f(n) = \mathcal{O}(g(n))$ if there exist positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.
- Meaning:** The algorithm will perform no worse than this upper limit.

2. Big Omega Notation (Ω): Lower Bound (Best Case)

- Definition:** $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$.
- Meaning:** The algorithm will perform no better (faster) than this floor.

3. Big Theta Notation (Θ): Tight Bound (Average/Exact Case)

- Definition:** $f(n) = \Theta(g(n))$ if $f(n) = \mathcal{O}(g(n))$ AND $f(n) = \Omega(g(n))$.
- Meaning:** The actual growth rate is tightly bounded above and below by $g(n)$.

Concrete Example: Linear Search vs. Binary Search

Consider searching for a target number inside an array of size n .

Algorithm	Best Case (Ω)	Worst Case ($\mathcal{O}$)	Tight Bound
Linear Search	$\Omega(1)$ (Target is 1st element)	$\mathcal{O}(n)$ (Target is last or missing)	Not applicable $\Theta(n)$ for worst case
Binary Search	$\Omega(1)$ (Target is middle element)	$\mathcal{O}(\log n)$ (Target at extreme end)	$\Theta(\log n)$ for all cases

...

Step-by-Step Breakdown of $f(n) = 3n^2 + 5n + 12$

To find the asymptotic complexity of an exact mathematical operation count:

1. **Identify terms:** $3n^2$, $5n$, and 12 .
2. **Keep the dominant term:** As $n \rightarrow \infty$, $3n^2$ grows far faster than $5n$ or 12 . Drop $5n + 12$.
3. **Drop constant coefficients:** Strip the factor of 3 .
4. **Final Complexity:** $\mathcal{O}(n^2)$ or $\Theta(n^2)$.

Common Growth Orders (Fastest to Slowest)

$$\mathcal{O}(1) < \mathcal{O}(\log n) < \mathcal{O}(n) < \mathcal{O}(n \log n) < \mathcal{O}(n^2) < \mathcal{O}(2^n) < \mathcal{O}(n!)$$

- **Constant ($\mathcal{O}(1)$):** Array lookup by index.
- **Logarithmic ($\mathcal{O}(\log n)$):** Binary search.
- **Linear ($\mathcal{O}(n)$):** Single pass over an array.
- **Linearithmic ($\mathcal{O}(n \log n)$):** Merge Sort, Quick Sort (average).
- **Quadratic ($\mathcal{O}(n^2)$):** Nested loops (Bubble Sort, Selection Sort).
- **Exponential ($\mathcal{O}(2^n)$):** Recursive Fibonacci without dynamic programming.

Define Asymptotic Notation, Explain Asymptotic Notation with Example, Write the application of Asymptotic Notation

Asymptotic Notation is a mathematical tool used to describe the limiting behavior and execution performance of an algorithm as its input size (n) approaches infinity. It ignores hardware differences, constant factors, and lower-order terms to focus strictly on how resource consumption (time or memory) grows with scale.

The Three Fundamental Types

- **Big O Notation ($\mathcal{O}$): Upper Bound** — Measures the worst-case scenario. It guarantees that the execution time will not exceed a given rate.
- **Big Omega Notation (Ω): Lower Bound** — Measures the best-case scenario. It defines the minimum runtime required by the algorithm.
- **Big Theta Notation (Θ): Tight Bound** — Represents the exact/average-case growth rate when the upper and lower bounds match.

Example: Analyzing an Algorithm

Consider a simple program that searches for an element in an array of size n , followed by a nested loop operation:

Python



```
def example_function(arr, target):  
    # Pass 1: Linear Search -> takes up to n steps  
    for x in arr:  
        if x == target:  
            break  
  
    # Pass 2: Print all pairs -> takes n * n steps  
    for i in arr:  
        for j in arr:  
            print(i, j)
```

Step-by-Step Derivation:

1. **Calculate exact operations:** $f(n) = n + n^2$
2. **Identify the dominant term:** As $n \rightarrow \infty$, n^2 grows drastically faster than n . Drop lower-order terms (n).
3. **Drop constant coefficients:** If the formula were $3n^2$, drop the multiplier 3.

4. Final Complexity: $\mathcal{O}(n^2)$

Key Applications

- **Algorithm Selection:** Enables developers to select the most efficient algorithm before writing any code (e.g., choosing QuickSort $\mathcal{O}(n \log n)$ over Bubble Sort $\mathcal{O}(n^2)$ for large datasets).
- **Performance Scalability:** Predicts how systems will handle load spikes as user traffic or data volume increases from thousands to millions of entries.
- **Hardware-Independent Benchmarking:** Allows researchers to evaluate algorithmic efficiency independently of CPU clock speeds, compiler optimizations, or programming languages.
- **Space-Time Trade-off Analysis:** Helps balance memory usage (Space Complexity) against execution speed (Time Complexity) when designing resource-constrained systems.

Where would you like to explore next?

Practice calculating time complexity on sample code snippets



Learn the formal mathematical proof for Big O bounds

